基于AI智能体的计算机软件源码漏洞检测方法研究

吕磊
（中电长城科技有限公司，杭州 310051）

摘　要：针对计算机软件源码漏洞检测误报率高、漏报率高及人工分析成本高等问题，开展了基于AI智能体的计算机软件源码漏洞检测方法研究。通过构建多层抽象语义表征模型，深度解析复杂代码结构并表征其脆弱性模式。研究基于AI智能体引入多智能体协同推理机制，借助意图规划、符号执行与环境交互，自主探索并验证漏洞触发路径。结合可微分程序分析技术，实现漏洞检测过程的可解释性与自适应优化。对比实验表明，所提方法在检测精度与泛化能力方面显著优于现有检测方法，能够有效识别潜在未知漏洞模式。

关键词：AI智能体；软件；漏洞；源码；计算机
DOI:10.16184/j.cnki.comprg.2026.09.041

1　概述

针对智能化漏洞检测的研究已取得诸多进展。包晟宏等[1]提出的集成式PU学习方法，创新性地从正样本与无标签样本中学习漏洞模式，有效缓解了实际场景中高质量漏洞样本稀缺的局面。不过，该方法本质上仍属于离线批处理学习范式，其检测过程具有单向和静态的特点。赵鹃等[2]致力于融合代码的多模态特征，以增强模型对源码的语义理解。这种多模态融合策略丰富了模型的输入信息层次，但该方法的模型架构依然是固定和封闭的，其推理过程不具备自主性和迭代性。综上所述，开展了基于AI智能体的计算机软件源码漏洞检测方法研究。

2　构建跨过程层次化语义表征模型

为克服传统代码表征方法在捕捉长距离依赖及跨函数语义联系方面的局限性，并为后续智能体决策提供精准的环境表征，设计了一种多层次、跨过程的源码语义表征模型[3]。该模型旨在将源代码从原始字符序列转化为富含语法、语义及数据流依赖信息的结构化图表示。该图表示可由图神经网络（GNN）处理，其核心由抽象语法图（ASG）与跨过程数据流图（IDFG）融合构成。

对于给定的目标程序，通过解析器生成其扩展的抽象语法图 $G_{asg}=(V_{asg},E_{asg})$。其中，节点集合 V_{asg} 不仅包含标准的AST节点类型，还特别引入了对安全分析至关重要的敏感操作节点类型。边集合 E_{asg} 则包含语法父子关系边 E_{syn} 和一种新定义的语义关联边 E_{sem}。语义关联边依据分析程序中的控制依赖和命名实体的关联性而构建，用于连接在语法上不直接相邻但在语义上紧密相关的节点，从而增强图的连通性。

为了精确刻画数据是如何在不同函数间传播的，特别是追踪污点的传播路径，构建了跨过程数据流图 $G_{idf_g}=(V_{df},E_{df})$。该图以数据为核心，$V_{df}$ 节点为变量的定义（definition）和使用（use）点，而边 E_{df} 则包括函数内数据流边和跨函数调用-参数传递边[4]。通过过程间分析，把各函数内的数据流图和函数间的调用关系连接起来，进而形成全局统一的数据流视图。

对 G_{asg} 与 G_{idf_g} 进行异构融合，形成一个统一的程序表征图 $G_{combined}=(V,E)$，其中，$V=V_{asg}\cup V_{df}$，$E=E_{asg}\cup E_{df}$。为了将此图转化为智能体可理解的向量空间表示，采用一个分层图注意力网络HGAT对其进行编码。编码过程在节点级别进行，如公式（1）所示：

$$h_i^{(l+1)}=\sigma\left(\sum_{j\in N(i)}\alpha_{ij}^{(l)}\boldsymbol{W}^{(l)}h_j^{(l)}\right)\tag{1}$$

其中，$h_j^{(l)}$ 为节点 i 在第 l 层的表示；$N(i)$ 为节点 i 的邻居集合；$\boldsymbol{W}^{(l)}$ 为可学习的权重矩阵；σ 为非线性激活函数。注意力系数 $\alpha_{ij}^{(l)}$ 计算了边 e_{ij} 的类型及其连接的两个节点特征的重要性，使得模型能够区分语法边、语义边和数据流边对表征的不同贡献。

在获得所有节点的最终表示后，设计了一个基于子图结构的层级池化操作，将代码表征从函数级汇聚到文件级，最终生成一个多粒度的、富含上下文信息的程序表征向量 H_{prog}，作为整个系统的基础环境状态表示。此表征不仅涵盖了代码的语法结构，还嵌入了其深层次的、跨过程的数据依赖语义，为智能体的后续推理提供了坚实的知识支撑。

3　基于AI智能体的漏洞触发路径自主探索

在获得程序的高维语义表征 H_{prog} 后，本模块的核心任务是模拟安全专家的分析思维，动态地、有目的地探索程序中可能存在的脆弱路径[5]。为实现这一目

作者简介：吕磊（1992—），男，本科，高级工程师，研究方向为人工智能。

中国知网　https://www.cnki.net

标，设计了由三类智能体组成的协同推理系统：规划器（Planner）、探索器（Explorer）和验证器（Verifier）。三者形成一个“决策—执行—验证”的闭环。

规划器负责从全局视角评估程序状态，并生成高层次的意图规划，其输入为当前的环境状态s_t（由H_{prog}和智能体历史动作共同决定），输出为一个目标函数g_t，该目标指示了当前最值得关注的潜在漏洞类型和代码区域。在具体应用中，探索从read函数到strcpy函数的未经验证的数据流路径。规划器的决策策略$\pi_p(a_p|s_t)$通过深度强化学习进行训练，其奖励函数设计为鼓励发现新的、复杂的代码状态转移。

探索器负责根据规划器设定的目标g_t，在具体的代码图谱G_{combined}上执行细粒度的路径探索。将探索任务建模为从起始节点（敏感源）到目标节点（危险汇聚点）的序列决策问题，并采用符号执行与启发式搜索相结合的方式[6]。探索器在每一步选择一个相邻节点（即一条代码边）进行游走，同时，维护一个符号执行环境ξ，记录路径约束条件，其选择节点的策略$\pi_e(a_e|s_t,g_t,\xi)$考虑了节点与目标的语义相似度、路径约束的复杂性，以及历史探索的覆盖度，以平衡探索与利用。

验证器负责对探索器发现的疑似漏洞路径进行最终确认。它接收一条由节点序列表示的候选路径P及其累积的路径约束Φ。验证器的核心任务是判断约束Φ的可满足性，并生成具体的触发输入，即POC概念[7]。设计了一个轻量级的可满足性模理论（SMT）求解器包装器，并结合基于学习的约束简化模型，以加速求解过程。如果Φ被判定为可满足，则验证器输出一个具体的漏洞报告，包含漏洞类型、位置和触发条件。

这3个智能体协同工作：规划器制定战略方向；探索器进行战术执行，在环境中留下足迹；验证器对关键发现进行裁决，并将结果反馈给规划器，以更新其对环境风险认知的状态s_{t+1}。

4 计算机软件源码漏洞自适应检测

为使上述多智能体系统能够端到端地学习并优化漏洞检测策略，引入了可微分程序分析技术，旨在对传统离散、符号化的程序分析关键步骤进行连续松弛，使梯度能够反向传播，从而驱动智能体策略自我进化。

对探索器在数据流图上的“离散节点选择”动作进行软化。在训练阶段，探索器输出一个在所有候选节点上的概率分布$p=\text{softmax}(z/\tau)$，其中，z为各节点的打分值；τ为温度系数。

通过运用Gumbel-softmax重参数化技巧，能够从这个分布中采样得到可微的近似离散动作，进而使得梯度可以借助选择节点这一动作，反向传播至上游的图编码网络和规划器网络。

针对验证器中的约束求解过程开展可微分近似。传统的SMT求解器属于离散的黑盒。设计了一个可微分约束评估网络（DCEN），其目标是学习预测一个约束表达式ϕ的可满足性倾向分数$y_\phi\in[0,1]$，以及满足条件情况下其解的大致分布。该网络将约束表达式解析为树结构，并使用递归神经网络进行编码，如公式（2）和公式（3）所示：

$$\boldsymbol{v}_\phi = f_\theta[\text{TreeRNN}(\phi)] \tag{2}$$

$$y_\phi = \sigma(w^T v_\phi + b) \tag{3}$$

其中，f_θ为递归神经网络；w和b为可学习参数；σ函数。在训练过程中，采用真实SMT求解器在采样约束上所得的结果作为监督信号，以此训练DCEN逼近求解器的行为。在智能体交互过程中，DCEN提供的连续、可导的y_ϕ可以作为一个即时的、可微分的奖励信号，指导探索器重点探索那些约束看起来更容易满足的路径。同时，规划器也能利用全局的约束可满足性预测调整其长期目标。

整个优化目标是一个复合的强化学习目标函数，它结合了最终漏洞发现的稀疏奖励和由可微分组件提供的密集中间奖励。通过端到端的训练，系统的各个部分（从表征编码、规划、探索到约束评估）被联合优化，共同趋向于最大化发现真实漏洞的长期回报。

5 对比实验

5.1 实验环境

实验在配备高性能计算单元的工作站上开展，可确保满足模型训练与复杂程序符号执行所需的算力。实验环境配置如表1所示。

表1 实验环境配置

序号	类别	配置规格
（1）	CPU	Intel Xeon Gold 6348 @ 2.60 GHz
（2）	GPU	NVIDIA A100-PCIE-80 GB
（3）	内存	512 GB DDR4 ECC
（4）	存储	4 TB NVMe SSD
（5）	操作系统	Ubuntu 22.04 LTS
（6）	深度学习框架	PyTorch 2.1.0, CUDA 12.1
（7）	程序分析工具链	Joern v2.0.214 ， Z3 SMT Solver v4.12.2

AgentVulDet模型涉及多个可训练模块，其超参数经过网格搜索与验证集调优确定。

5.2 参数配置

在上述实验环境中，分别利用提出的检测方法、基于集成式PU学习的检测方法和基于多模态特征融合的检测方法对计算机软件源码漏洞进行检测。AgentVulDet关键参数配置如表2所示。

中国知网 https://www.cnki.net

表2　AgentVulDet关键参数配置

参数模块	参数名称	参数值
图编码器	节点嵌入维度	256
	图注意力层数	3
	注意力头数	8
	输出表征维度	512
多智能体训练	算法	PPO
	学习率(LR)	3×10^{-5}
	折扣因子	0.99
	熵正则化系数	0.01
探索器	最大路径深度	50

其中，图编码器采用3层图注意力网络，智能体训练采用近端策略优化（PPO）算法，因其在处理连续-离散混合动作空间上具有稳定性。

实验采用两个公开的、广泛使用的源代码漏洞基准数据集：Juliet Test Suite v1.3 (C/C++)与Draper VDISC (C/C++)。Juliet数据集包含大量具有已知脆弱模式的、小型但结构化的测试用例，适合评估方法对基础漏洞模式的识别能力。VDISC数据集则源自真实世界开源项目（例如，Linux Kernel、Wireshark）的函数片段，并经过人工标注，更具挑战性，可用于评估方法的泛化能力与对复杂漏洞的检测效果。将两个数据集混合，并按照项目来源进行划分，确保训练集、验证集和测试集的项目互斥，以防止数据泄露。最终，测试集包含超过15 000个函数样本，涵盖缓冲区溢出、整数溢出、格式化字符串漏洞等八大类CWE（通用弱点枚举）漏洞类型。

5.3　实验结果对比分析

为验证AgentVulDet在识别“潜在未知漏洞模式”和复杂漏洞方面的优势，从测试集中筛选出两类挑战性样本子集。跨过程漏洞：需要追踪数据流穿越至少3个函数调用的漏洞；路径约束复杂漏洞：触发条件涉及非线性运算或混合类型约束的漏洞。各方法在此类样本上的召回率对比如图1所示。

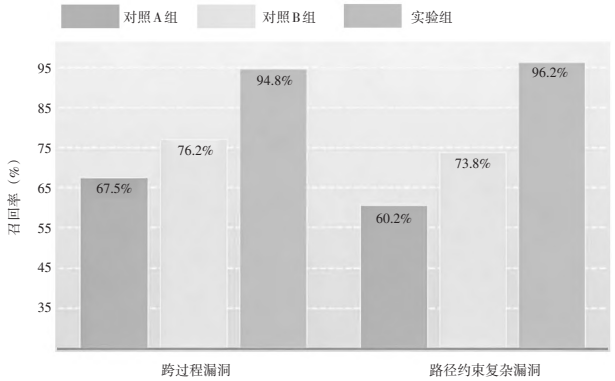


图1　各方法在此类样本上的召回率对比

由图1可知，对于跨过程漏洞，实验组的召回率远高于对照A组和对照B组。这直接证实了跨过程数据流图IDFG和多智能体协同探索具有关键作用。IDFG为探索器提供了跨函数的导航地图，而规划器与探索器的配合使其能够执行长周期的探索任务，克服了传统方法因分析范围局限而导致的失效。

6　结语

研究针对计算机软件源码漏洞检测问题，提出了一种基于AI智能体的创新方法。通过构建一个能够理解不同功能间数据流动的模型，智能体获取了大量有用的背景信息。研究不仅为检测软件漏洞提供了新方法，还为AI在程序分析中的应用打下了基础。未来，会继续优化模型，扩大应用范围，为软件安全做出更多贡献。

参考文献

[1] 包晟宏, 姚有健, 李小丫, 等. 集成式PU学习方法PUEVD及其在软件源码漏洞检测中的应用[J]. 计算机科学, 2025, 52(S1): 853-861.

[2] 赵鹃, 何成万, 叶庭瑞, 等. 基于多模态特征融合的源代码漏洞检测[J]. 计算机工程与设计, 2024, 45(11): 3233-3239.

[3] 陈子雄, 陈旭, 景永俊, 等. 基于图神经网络的源代码漏洞检测研究综述[J]. 计算机工程与科学, 2024, 46(10): 1775-1792.

[4] 曹炳豪, 汪智超, 朱二周. 面向软件漏洞检测的改进代码属性图的图神经网络[J]. 微电子学与计算机, 2024, 41(1): 74-82.

[5] 李秋月, 韩道军, 张磊, 等. 基于分层注意力网络和积分梯度的细粒度漏洞检测方法[J]. 计算机科学, 2024, 51(12): 326-333.

[6] 顾锡国, 王志伟, 陈翔, 等. SCG-Detector: 基于图注意力网络的智能合约漏洞检测方法[J]. 电子学报, 2024, 52(12): 4101-4112.

[7] 李浴淑, 邢颖, 陆思奇, 等. 基于深度学习的函数体切片级C/C++智能合约漏洞检测工具[J]. 计算机应用, 2025, 45(11): 3493-3501.

中国知网　https://www.cnki.net